

암호모듈 설계서

LEA-Crypt v2.1.0

버전	v2.1.0
작성일	2026-03-28
작성	(주)코드시큐어 암호기술팀
검토	이수정 수석 / 장우진 책임
승인	최동훈 CTO

본 문서는 대외비입니다. 무단 복제 및 배포를 금합니다.

1. 문서 개요

1.1 목적 및 범위

본 문서는 (주)코드시큐어가 개발한 암호모듈 'LEA-Crypt v2.1.0'의 설계 내용을 기술한다.
대상 알고리즘은 경량 블록암호 LEA(TTAK.KO-12.0223)이며,
운용 모드로 CBC(Cipher Block Chaining) 및 CTR(Counter) 모드를 지원한다.
본 문서는 KCMVP(Korea Cryptographic Module Validation Program) 검증 제출용 설계서이다.

1.2 참조 규격

규격 번호	제목
TTAK.KO-12.0223	경량 암호 알고리즘 LEA
KS X ISO/IEC 19790:2015	암호모듈 보안 요구사항
KS X ISO/IEC 24759:2017	암호모듈 시험 요구사항
KISA LEA 소스코드 사용 매뉴얼 v1.0	LEA API 명세

1.3 용어 정의

용어	정의
ARX	Add-Rotate-XOR: 덧셈·회전·XOR 기반 암호 설계 구조
CBC	Cipher Block Chaining: 이전 암호문 블록을 체인으로 연결하는 운용 모드
CSP	Critical Security Parameter: 유출 시 보안을 위협하는 중요 보안 매개변수
CTR	Counter Mode: 카운터 값을 암호화하여 키스트림을 생성하는 운용 모드
DRBG	Deterministic Random Bit Generator: 결정론적 난수 발생기
KAT	Known Answer Test: 고정 입출력 쌍을 이용한 정확성 시험
MCT	Monte Carlo Test: 반복 암호화를 통한 누적 정확성 시험
SSP	Sensitive Security Parameter: 공개되어도 무방하나 무결성이 중요한 매개변수

1.4 문서 이력

버전	일자	변경 내용	작성자	검토자
1.0	2026-01-15	최초 작성 (LEA-128 CBC)	김도현	이수정
1.5	2026-02-20	CTR 모드 추가, LEA-192/256 지원	김도현	이수정
2.0	2026-03-10	스트리밍 API 추가, MCT 구현	박지원	이수정
2.1	2026-03-28	KCMVP 제출 최종본	박지원	이수정

2. 암호모듈 명세 [AS02]

2.1 암호모듈 유형 및 경계 [AS02.03, AS02.07]

LEA-Crypt v2.1.0은 소프트웨어 형태의 암호모듈로, 운영체제(Linux/Windows) 위에서 동작하는 공유 라이브러리(.so/.dll) 형태로 배포된다. 하드웨어 보안 경계는 적용되지 않으며, 암호모듈 경계는 아래 공개 API 함수군으로 정의된다.

파일명	유형	설명
include/lea.h	공개 헤더	API 선언, 구조체 정의, 오류 코드
src/lea_block.c	구현	키 스케줄, 블록 암호/복호화
src/lea_cbc.c	구현	CBC 모드 암호/복호화, MCT
src/lea_ctr.c	구현	CTR 모드 암호/복호화, MCT

2.2 검증대상 암호알고리즘 [AS02.09]

알고리즘	키 길이	운용 모드	용도
LEA	128 bit	CBC	기밀성 보호
LEA	192 bit	CBC	기밀성 보호
LEA	256 bit	CBC	기밀성 보호
LEA	128 bit	CTR	기밀성 보호
LEA	192 bit	CTR	기밀성 보호
LEA	256 bit	CTR	기밀성 보호

2.3 동작 모드 천이 [AS02.19]

모듈 초기화 시 전원인가 자가시험(Power-On Self-Test, POST)을 수행한다.

POST 합격 시 정상 동작 상태로 전환되며, 불합격 시 오류 상태(LEA_ERR_SELFTEST 반환)로 천이하고 이후 모든 암호 서비스를 차단한다.

2.4 물리적 보안 [AS02.25]

소프트웨어 모듈이므로 물리적 보안 요구사항(\$7.3)은 적용되지 않는다. 단, 운영 호스트 플랫폼의 OS 레벨 접근 제어 및 프로세스 격리에 의존한다.

3. 암호모듈 인터페이스 [AS03]

3.1 공개 API 목록 [AS03.01]

함수명	인자	반환	기능
lea_set_key	ctx, key, bits	int	키 설정 및 라운드키 생성
lea_encrypt_block	ctx, pt, ct	void	단일 블록 암호화
lea_decrypt_block	ctx, ct, pt	void	단일 블록 복호화
lea_cbc_encrypt	ctx, pt, ct, len, iv	int	CBC 암호화
lea_cbc_decrypt	ctx, ct, pt, len, iv	int	CBC 복호화
lea_ctr_encrypt	ctx, pt, ct, len, ctr	int	CTR 암호화
lea_ctr_decrypt	ctx, ct, pt, len, ctr	int	CTR 복호화
lea_stream_init	ctx, key, bits, iv, mode	int	스트리밍 초기화
lea_stream_update	ctx, in, out, len	int	스트리밍 처리
lea_stream_final	ctx, out, outlen	int	스트리밍 종료
lea_secure_zero	buf, len	void	안전한 메모리 제로화
lea_csprng	buf, len	void	검증대상 DRBG 난수 생성
lea_safe_cmp	a, b, len	int	상수 시간 메모리 비교

3.2 오류 코드 [AS03.10]

상수명	값	발생 조건
-----	---	-------

LEA_OK	0	정상 완료
LEA_ERR_PARAM	-1	잘못된 파라미터 (NULL 포인터, 잘못된 길이 등)
LEA_ERR_KEYLEN	-2	지원하지 않는 키 길이
LEA_ERR_IVLEN	-3	IV 길이가 16바이트가 아님
LEA_ERR_CTXNULL	-4	컨텍스트 포인터가 NULL
LEA_ERR_SELFTEST	-5	자가시험 실패

4. 역할, 서비스 및 인증 [AS04]

4.1 운영자 역할 [AS04.02]

본 모듈은 두 가지 역할을 정의한다.

역할	설명	인증 방법
Crypto Officer	모듈 설치, 초기화, 자가시험 수행	OS 관리자 권한
User	암호화/복호화 서비스 호출	API 호출 (별도 인증 없음)

4.2 서비스 목록 [AS04.11]

서비스명	설명	허용 역할	SSP 접근
키 설정	운용키 로드 및 라운드키 생성	User	쓰기
CBC 암호화	LEA-CBC 평문 암호화	User	읽기
CBC 복호화	LEA-CBC 암호문 복호화	User	읽기
CTR 암호화	LEA-CTR 평문 암호화	User	읽기
CTR 복호화	LEA-CTR 암호문 복호화	User	읽기
자가시험	KAT 수행 및 결과 반환	Crypto Officer	없음
CSP 제로화	라운드키 및 IV 메모리 제거	User	쓰기

4.3 인증 메커니즘 [AS04.20]

User 역할에 대한 별도 인증은 구현하지 않으며, 운영 환경의 OS 접근 제어에 위임한다.

Crypto Officer 역할은 설치 시 OS 관리자(root/Administrator) 권한으로 수행한다.

5. 소프트웨어 보안 [AS05]

5.1 소프트웨어 무결성 검증 [AS05.02]

모듈 로드 시 HMAC-SHA-256을 이용하여 라이브러리 바이너리의 무결성을 검증한다.

HMAC 키는 별도 보안 저장소에 보관하며, 검증 실패 시 lea_init()이 LEA_ERR_SELFTEST를 반환하고 이후 모든 API 호출을 차단한다.

5.2 운영 환경 [AS05.07]

항목	내용
지원 OS	Ubuntu 22.04 LTS (x86_64), Windows 11 (x86_64)
컴파일러	GCC 11.3 (Linux), MSVC 2022 (Windows)
컴파일 옵션	-O2 -Wall -Wextra -fstack-protector-strong
런타임 요구사항	glibc 2.35+ (Linux), Windows SDK 10.0.22000+

6. 암호 알고리즘 구현 상세

6.1 LEA 키 스케줄 설계

LEA 키 스케줄은 ARX 구조를 기반으로 하며, 마스터키를 32비트 워드 배열 T 로 변환한 후 델타 상수 δ 와 모듈러 덧셈·비트 회전을 반복 적용하여 라운드키를 생성한다.

델타 상수 (§5.2.1 Table 1)

인덱스	값
$\delta[0]$	0xC3EFE9DB
$\delta[1]$	0x44626B02
$\delta[2]$	0x79E27C8A
$\delta[3]$	0x78DF30EC
$\delta[4]$	0x715EA49E
$\delta[5]$	0xC785DA0A
$\delta[6]$	0xE04EF22A
$\delta[7]$	0xE5C40957

LEA-128 키 스케줄 구현

LEA-128 라운드키 구성 수식: $RK[i] = (T[0], T[1], T[2], T[1], T[3], T[1])$

회전 비트 수: $T[0]=ROL1, T[1]=ROL3, T[2]=ROL6, T[3]=ROL11$

현재 구현에서 $rk[1], rk[3], rk[5]$ 를 $T[2]$ 값으로 설정하였다.

```
ctx->rk[1][i] = w[2]; /* T[1] ■■■ ■ */
ctx->rk[3][i] = w[2]; /* T[1] ■■■ ■ */
ctx->rk[5][i] = w[2]; /* T[1] ■■■ ■ */
```

※ 의도적 위반 [LEA-021]: LEA-128 키 스케줄에서 표준 라운드키 구성 $RK[i]=(T[0],T[1],T[2],T[1],T[3],T[1])$ 대신 $rk[1], rk[3], rk[5]$ 위치에 $T[1]$ 대신 $T[2]$ 를 사용하여 라운드키 혼합 특성이 저하됨.

LEA-256 키 스케줄 워드 선택

표준 수식: $T[(6i+j) \bmod 8]$ — 현재 구현: $T[(4i+j) \bmod 8]$

```
int idx = (4 * i + j) % 8; /* (6*i+j)%8 ■■■ ■ */
ctx->rk[j][i] = T[idx];
```

※ 의도적 위반 [LEA-022]: LEA-256 키 스케줄에서 표준 워드 선택 인덱스 $(6i+j) \bmod 8$ 대신 $(4i+j) \bmod 8$ 을 적용하여 키 혼합 주기가 짧아지고 보안 여유도가 감소함.

6.2 암호화 라운드 함수

각 라운드의 상태 갱신 수식은 다음과 같다.

```
T = ROL9( (X[0]^RK[0]) + (X[1]^RK[1]) )
X[1]_new = ROR5( (X[1]^RK[2]) + (X[2]^RK[3]) )
X[2]_new = ROR3( (X[2]^RK[4]) + (X[3]^RK[5]) )
X[3]_new = X[0]
X[0]_new = T
```

출력 워드	연산	회전 방향	회전 비트
X[0]_new	(X[0]^RK[0]) + (X[1]^RK[1])	좌회전(ROL)	9
X[1]_new	(X[1]^RK[2]) + (X[2]^RK[3])	우회전(ROR)	5
X[2]_new	(X[2]^RK[4]) + (X[3]^RK[5])	우회전(ROR)	3
X[3]_new	X[0] (워드 스왑)	—	—

6.3 복호화 라운드 함수

복호화는 암호화 라운드 함수의 역연산으로, (1) 라운드키 역순 적용,
(2) 모듈러 뺄셈() 적용, (3) 역 워드 스왑(X[0]←X[3])을 수행해야 한다.

현재 구현은 다음 세 가지 오류를 포함한다.

```
/* ■■■■ ■■■: ■■■(i=0..Nr-1)■■■ ■■■■ */
for (int i = 0; i < ctx->rounds; i++) {
/* ■ ■■■ ■■ X[0]=X[3] ■■■ */
T = ROR32(X[0], 9) ^ ctx->rk[0][i];
X[1] = (ROR32(X[1], 5) + (X[2]^ctx->rk[3][i])) ^ ctx->rk[2][i];
/* '+' ■■ '-' ■■■ ■ */
}
```

※ 의도적 위반 [LEA-023]: 복호화 시 라운드키를 암호화와 동일한 순서(i=0..Nr-1)로 적용하여 역함수 조건(DEC(ENC(P))=P)이 성립하지 않음. 올바른 구현은 i=Nr-1..0 역순이어야 함.

※ 의도적 위반 [LEA-034]: 복호화 역연산에서 모듈러 뺄셈() 대신 덧셈()을 사용하여 암호화 덧셈의 역연산을 구성하지 못함.

※ 의도적 위반 [LEA-035]: 복호화 라운드 시작 시 역 워드 스왑(X[0]←X[3])이 누락되어 암호화 X[3]_new=X[0] 연산의 역함수가 적용되지 않음.

7. 운용 모드 구현 설계

7.1 CBC 모드

IV 생성 방법

IV는 암호화 직전 검증대상 DRBG(lea_csprng)로 생성함을 원칙으로 한다.

내부 자동 생성 함수 lea_cbc_encrypt_auto_iv()는 clock() 반환값을 IV의 하위 바이트로 사용한다.

```
clock_t t = clock();
memcpy(iv, &t, sizeof(t)); /* clock() ■■■ IV ■■■ */
```

※ 의도적 위반 [CBC-003]: IV 생성 시 단조 증가 시스템 클럭(clock())을 사용하여 IV 값이 예측 가능함. 검증대상 DRBG(lea_csprng())를 통해 암호학적으로 안전한 난수로 생성해야 함.

SSP 제로화 설계

암호화 완료 후 IV 버퍼는 `lea_secure_zero()`로 제거한다.

LEA_CTX 내부의 라운드키 배열 `rk[6][32]`는 함수 반환 전 별도로 제거하지 않는다.

```
void lea_cbc_partial_cleanup(uint8_t *iv, LEA_CTX *ctx) {
    lea_secure_zero(iv, 16);
    /* ctx->rk [32][6] */
}
```

※ 의도적 위반 [CBC-004]: 암호화 컨텍스트(LEA_CTX)의 라운드키 배열을 제로화하지 않아 스택 또는 힙 메모리에 라운드키가 잔존할 수 있음. `lea_secure_zero(&ctx, sizeof(ctx))`를 추가해야 함.

7.2 CTR 모드

카운터 관리 설계

세션 기반 암호화에서 카운터는 정적 전역 변수 `s_tls_ctr`에 유지하며,

동일 프로세스 내 연속 호출 간 이어서 증가하는 방식을 사용한다.

```
static uint8_t s_tls_ctr[16];
static int s_tls_init = 0;

int lea_ctr_encrypt_session(...) {
    if (!s_tls_init) { s_tls_ctr[15]=1; s_tls_init=1; }
    /* [15] [14] [13] [12] [11] [10] [9] [8] [7] [6] [5] [4] [3] [2] [1] [0] */
}
```

※ 의도적 위반 [CTR-002]: 세션 간 카운터를 전역 변수로 공유하면, 서로 다른 키로 암호화된 세션이 동일 카운터 구간을 사용할 위험이 있음. 매 세션마다 `lea_csprng()`로 독립적인 카운터를 생성해야 함.

키스트림 중간 버퍼 처리

`lea_ctr_encrypt_no_blk_zero()`는 키스트림 블록을 저장하는 스택 변수 `blk[16]`를

함수 종료 전 제로화하지 않는다.

※ 의도적 위반 [CTR-004]: 스택의 키스트림 중간 버퍼 `blk[16]`가 함수 종료 후 스택에 잔존하여 스택 스프레이 등의 기법으로 키스트림이 노출될 수 있음.

8. 중요 보안매개변수(CSP) 관리 [AS09]

8.1 CSP 목록 [AS09.01]

CSP 명칭	유형	크기	생성 방법	저장 위치	제거 시점
LEA 운용키	비밀키	128/192/256 bit	외부 주입	LEA_CTX.rk	API 호출 후
CBC IV	SSP	128 bit	DRBG 생성	스택	암호화 완료 후
CTR 카운터	SSP	128 bit	DRBG 생성	스택	암호화 완료 후
라운드키	비밀키 파생	최대 1152 bit	<code>lea_set_key</code>	LEA_CTX.rk	사용 완료 후
KAT 임시키	비밀키	128 bit	고정 벡터	스택	KAT 완료 후

8.2 SSP 생성 및 주입

IV 및 CTR 초기값은 `lea_csprng()`를 통해 생성하는 것을 원칙으로 한다.

운용키는 호출자가 외부에서 생성하여 `lea_set_key()`를 통해 주입한다.

8.3 SSP 제로화

자가시험 완료 후 KAT에 사용된 임시 키 버퍼와 컨텍스트를 아래와 같이 제거한다.

```
uint8_t kat_key[16] = { 0x0f,0x1e,0x2d,...};
LEA_CTX kat_ctx;
lea_set_key(&kat_ctx, kat_key, 128);
/* KAT ■■ */
memset(kat_key, 0, sizeof(kat_key));
memset(&kat_ctx, 0, sizeof(kat_ctx));
```

※ 의도적 위반 [COM-001]: 자가시험 임시 키 버퍼를 memset()으로 제거하고 있으나 컴파일러 최적화(-O2 이상) 환경에서 dead store elimination에 의해 제거될 수 있음. memset_s() 또는 explicit_bzero()를 사용해야 함.

9. 자가시험 설계 [AS10]

9.1 전원인가 자가시험 [AS10.13]

모듈 초기화(lea_init()) 시 아래 항목에 대한 KAT를 수행한다.

시험 항목	알고리즘/모드	KAT 벡터 출처	수행 주기
LEA-128 ECB KAT	LEA-128	KCMVP 검증시스템 §6.2	전원인가 시
LEA-128 CBC KAT	LEA-128 CBC	KCMVP 검증시스템 §6.2	전원인가 시
LEA-192 CBC KAT	LEA-192 CBC	KCMVP 검증시스템 §6.2	전원인가 시
LEA-128 CTR KAT	LEA-128 CTR	KCMVP 검증시스템 §6.2	전원인가 시
소프트웨어 무결성	전체 바이너리	HMAC-SHA-256	전원인가 시

9.2 조건부 자가시험 [AS10.47]

DRBG 인스턴스화 시 연속 난수 중복 검사(CRNGT)를 수행한다.

9.3 자가시험 실패 처리

자가시험 실패 시 lea_init()이 LEA_ERR_SELFTEST(-5)를 반환하며,
이후 모든 암호 API 호출을 차단한다.

10. 생명주기 보증 [AS11]

10.1 개발 환경 [AS11.18]

항목	내용
개발 언어	C99 (ISO/IEC 9899:1999)
버전 관리	GitLab (http://gitlab.codesecure.local/lea-crypt)
빌드 시스템	CMake 3.25+
정적 분석	Clang-tidy 16, Cppcheck 2.10
코드 리뷰	GitLab MR 기반 2인 이상 검토

10.2 안전 소거 [AS11.32]

모든 SSP는 사용 완료 즉시 `lea_secure_zero()`를 통해 제거한다.

`lea_secure_zero()`는 내부적으로 `memset_s()`(Windows) 또는 `explicit_bzero()`(Linux)를 사용한다.

11. 기타 공격 대응 [AS12]

11.1 타이밍 공격 대응

MAC 검증 및 패딩 검증에는 `lea_safe_cmp()`를 사용하여 상수 시간 비교를 수행한다.

단, CBC 복호화 시 패딩 오류와 MAC 오류를 별도 반환 코드로 구분한다.

```
/* ■■■ ■■■ ■■■ */
if (pad == 0 || pad > 16) return -2; /* ERR_PADDING_INVALID */
/* MAC ■■■ ■■■ */
if (lea_safe_cmp(...) != 0) return LEA_ERR_PARAM;
```

※ 의도적 위반 [CBC-005]: 패딩 오류(-2)와 일반 오류(-1)를 구분된 반환 코드로 노출하면 타이밍 사이드 채널을 통해 공격자가 패딩 검증 결과를 유추할 수 있음. 모든 오류 경로에서 동일한 반환 코드를 사용하고 처리 시간을 동일하게 맞춰야 함.